WDJ - QB UNIT 6 (5- marks)

1 Define ReactJS and explain its features.

1. ReactJS is a JavaScript library used for building user interfaces and single-page applications. It was developed by Facebook (Meta) and uses a component-based architecture.
2. One important feature of ReactJS is component-based development. Components are reusable UI blocks that help organize applications into smaller parts.
3. ReactJS uses a Virtual DOM to improve application performance. The Virtual DOM updates only changed elements instead of updating the complete real DOM.
4. ReactJS supports reusable code which saves development time and effort. Developers can use the same component multiple times in different parts of the application.
5. ReactJS provides fast rendering and improves user experience in web applications. It updates the UI quickly whenever data changes in the application.
6. ReactJS is easy to learn and supports one-way data binding. One-way data binding helps control data flow and makes debugging easier.

2 What is JSX? Explain its advantages.

1. JSX stands for JavaScript XML and is used in React applications. It allows developers to write HTML inside JavaScript code easily.
2. JSX makes UI code simple, readable, and easy to understand. It combines JavaScript and HTML syntax together in one file.
3. JSX helps developers create React elements without using createElement() or appendChild() methods. This reduces code complexity and improves coding speed.
4. JSX supports JavaScript expressions using curly braces {} inside HTML code. This allows dynamic data to be displayed easily in the user interface.
5. JSX follows simple rules such as using one parent element and closing all tags properly. It also uses className instead of class while writing HTML attributes.
6. JSX converts HTML-like code into React elements automatically. This helps React render content efficiently inside the browser DOM.

3 Explain React component architecture.

1. React component architecture divides the user interface into smaller reusable components. Each component represents a separate part of the UI such as a header or footer.
2. Components help developers organize code in a clean and structured way. This architecture improves readability and makes maintenance easier.
3. React provides two types of components called class components and functional components. Both types are used to create and display UI elements in applications.
4. Components can communicate with each other using props. Props allow data to pass from parent components to child components.
5. State is used inside components to store dynamic data and update the UI. Whenever the state changes, React automatically re-renders the component.
6. Component architecture supports code reusability and faster development. Developers can reuse the same component in multiple places within the application.

4 What are class components in React?

1. Class components are React components written using ES6 classes. They must extend React.Component to use React features.
2. A class component uses a render() method to display content on the screen. Without the return statement inside render(), nothing will be shown in the browser.
3. Class components can store and manage data using state. State contains dynamic data that can change during application execution.
4. Event handling can also be implemented inside class components. Functions like increment() can update state values using setState().
5. Class components support props for passing data from parent to child components. Props are read-only and accessed using this.props.
6. Earlier, class components were mainly used for state and lifecycle features in React. Now hooks allow functional components to perform many similar tasks.

5 Differentiate between state and props.

1. Props are used to pass data from parent components to child components in React. State is used to store dynamic data inside a component.
2. Props are read-only and cannot be modified by the child component. State is mutable and can be updated whenever needed.
3. Props help in component communication between parent and child components. State is managed within the same component where it is created.
4. Props are passed from one component to another using attributes. State values are updated using setState() or useState().
5. Props do not directly control component behavior after being passed. State changes automatically re-render the user interface in React.
6. Props improve data sharing and component reusability in applications. State helps manage changing data like counters, forms, and user input.

6 What is state in React?

1. State in React is used to store dynamic data inside a component. It keeps information that can change during application execution.
2. State can be updated whenever required in a React component. When the state changes, the UI automatically re-renders with updated data.
3. In class components, state is updated using the setState() method. This method changes the stored data and refreshes the interface.
4. In functional components, the useState hook is used for state management. It stores data and updates the UI automatically.
5. State is useful for handling counters, form values, and user interactions. It helps applications respond dynamically to user actions.
6. State is managed within the component itself and is not read-only. This makes React applications interactive and responsive.

7 What are props?

1. Props are used in React to pass data from a parent component to a child component. They help components communicate with each other.
2. Props are read-only values and cannot be modified by child components. This helps maintain a controlled flow of data in the application.
3. Props are passed using attributes while calling a component. These values can then be accessed inside the child component.
4. In class components, props are accessed using this.props. This allows components to display and use received data easily.
5. Props help make components reusable because different data can be passed each time. The same component can display different outputs using different props.
6. Props improve component communication and application structure in React. They are commonly used for sending names, values, and configuration data.

8 Explain event handling in React.

1. Event handling in React is used to respond to user actions like clicks and typing. React handles events using special event attributes.
2. React uses camelCase syntax for event names such as onClick and onChange. These events are attached directly to HTML elements in JSX.
3. Functions are created to handle events when they occur in the application. For example, clicking a button can call a function that shows an alert message.
4. Event handling can also update component state using setState() or useState(). This allows the UI to change dynamically based on user actions.
5. Forms in React use events like onChange and onSubmit for handling user input. The event.target.value property is used to get entered data from input fields.
6. React event handling helps create interactive and responsive applications. It improves user experience by reacting quickly to user actions.

9 What is conditional rendering?

1. Conditional rendering in React is used to display UI based on certain conditions. Different content is shown depending on whether the condition is true or false.
2. React supports conditional rendering using if statements inside components. The program checks conditions and displays suitable output to users.
3. The ternary operator can also be used for conditional rendering in React. It provides a shorter and cleaner way to display different UI elements.
4. React also supports the && operator for showing elements when conditions are true. This method is useful for displaying small UI sections conditionally.
5. Conditional rendering improves user experience by displaying proper messages and content. For example, users can see “Welcome User” or “Please Login” messages.
6. This feature helps developers create dynamic and interactive applications in React. It allows the interface to change automatically according to application data.

10 Explain lists in React.

1. A list in React is a collection of items usually stored in an array. Lists are used to display dynamic data in applications.
2. React uses arrays and the map() function to display list items dynamically. The map() function loops through each element in the array.
3. Each item in a React list should have a unique key attribute. The key helps React identify elements efficiently during updates.
4. Lists are useful for displaying records such as student names, products, or messages. They help create reusable and dynamic user interfaces.
5. In React, the map() function creates elements like <li> for every array item. The displayed output changes automatically if the array data changes.
6. React lists improve code simplicity and make data handling easier in applications. They are commonly used in modern web interfaces for displaying multiple records.

11 How are forms handled in React?

1. Forms in React are used to collect user input such as names and login details. Common form elements include input fields, textarea, dropdowns, and buttons.
2. React handles form data using event handling methods like onChange and onSubmit. These events help capture and process user input values.
3. The event.target.value property is used to get values entered by the user. The event.preventDefault() method stops page refresh during form submission.
4. React forms can use uncontrolled components where the DOM manages the data directly. In this approach, form values are taken from input fields during submission.
5. React also supports controlled components where form data is managed using state. The input value is connected to state using value and onChange attributes.
6. Form handling in React keeps the UI and data synchronized automatically. This helps developers build interactive and user-friendly applications.

12 What are React Hooks?

1. React Hooks are special functions that allow functional components to use state and other React features. Hooks made functional components more powerful and flexible.
2. Earlier, only class components could use state and lifecycle features in React. Hooks now allow functional components to perform the same tasks easily.
3. One common hook is useState which is used to store and update data in functional components. It automatically updates the UI whenever state changes.
4. Another important hook is useEffect which is used for side effects in components. It runs after rendering and is commonly used for API calls and timers.
5. Hooks simplify state management and reduce the need for class components. They make React code shorter, cleaner, and easier to understand.
6. Hooks improve code reusability and application performance in React projects. They are widely used in modern React application development.

13 Explain useState hook.

1. useState is a React Hook used to store and update data in functional components. It allows functional components to manage state easily.
2. The syntax of useState is const [state, setState] = useState(initialValue);. Here, state stores data and setState updates the value.
3. useState helps React remember values entered or changed by the user. It is commonly used for counters, forms, and dynamic UI data.
4. When the state value changes, React automatically updates the user interface. This makes applications interactive and responsive.
5. useState replaces the need for state management in class components. It makes functional components simpler and easier to understand.
6. The useState hook improves code readability and reduces complexity in React applications. It is one of the most commonly used hooks in React development.

14 Explain useEffect hook.

1. useEffect is a React Hook used for performing side effects in components. It runs after the component renders on the screen.
2. This hook is commonly used for API calls, DOM updates, and timers in React applications. It helps perform actions automatically when components load.
3. The basic syntax of useEffect is useEffect(() => { }, []);. The empty dependency array means the code runs only once when the page loads.
4. useEffect is often used with fetch() to get data from backend APIs. The fetched data can then be stored using useState.
5. In React applications, useEffect helps automatically load data from servers when the component starts. This improves automation and user experience.
6. The useEffect hook simplifies lifecycle-related tasks in functional components. It reduces the need for class component lifecycle methods.

15 List advantages of using ReactJS.

1. ReactJS uses component-based architecture for building applications. Components are reusable and help organize the UI into smaller parts.
2. ReactJS uses a Virtual DOM which improves application performance. Only changed elements are updated in the real DOM.
3. ReactJS supports reusable code which saves development time and effort. Developers can use the same components multiple times in applications.
4. ReactJS provides fast rendering and smooth UI updates for users. This improves the overall user experience of web applications.
5. ReactJS is easy to learn because it uses simple syntax and JSX. Developers can create readable and maintainable code easily.
6. ReactJS supports one-way data binding which makes data flow easier to manage. This helps in debugging and maintaining applications properly.

(10 – marks)

1 Explain ReactJS features, architecture, and advantages.

1. ReactJS is a JavaScript library used for building user interfaces and single-page applications. It was developed by Facebook (Meta) to create fast and interactive web applications.
2. ReactJS uses component-based architecture where the user interface is divided into smaller reusable components. Components like header, footer, and student cards can be reused in multiple parts of an application.
3. ReactJS provides a Virtual DOM which is a lightweight copy of the real DOM. React updates the Virtual DOM first and changes only the required elements in the real DOM for better performance.
4. ReactJS supports reusable code which reduces development time and improves efficiency. Developers can use the same component many times instead of writing repeated code.
5. ReactJS supports one-way data binding which controls the flow of data inside applications. This makes applications easier to debug and maintain properly.
6. ReactJS architecture uses props and state for handling data in components. Props are used for passing data while state stores dynamic data inside components.
7. ReactJS provides fast rendering and automatic UI updates when data changes. This improves the overall user experience and application speed.
8. ReactJS is easy to learn because it uses simple syntax and JSX for writing UI code. Developers can create readable, maintainable, and scalable applications easily.
9. ReactJS also supports hooks like useState and useEffect in functional components. Hooks simplify state management and make React applications more efficient.

2 Explain JSX in detail with syntax and examples.

1. JSX stands for JavaScript XML and is an important feature of ReactJS. It allows developers to write HTML-like code inside JavaScript files easily.
2. JSX makes the user interface code simple, readable, and easy to understand. It combines JavaScript and HTML syntax together in one place.
3. JSX syntax looks similar to HTML but it is actually JavaScript code. React converts JSX into React elements before displaying them in the browser.
4. A simple JSX example is return <h1>Hello</h1>; which displays a heading on the webpage. JSX can also create lists, tables, and forms directly inside JavaScript.
5. JSX supports JavaScript expressions inside curly braces {} for dynamic content. Developers can display variables, calculations, and function results inside the UI.
6. JSX follows some important rules while writing code in React applications. It requires one parent element, proper closing tags, and uses className instead of class.
7. JSX removes the need for methods like createElement() and appendChild() in JavaScript. This reduces coding complexity and makes development faster and easier.
8. JSX allows developers to build reusable and dynamic UI components efficiently. It improves application structure and helps manage large React projects properly.
9. JSX is widely used in React because it improves coding speed and readability. It helps developers create interactive and modern web applications with less effort.

3 Explain React components and component architecture with examples.

1. React components are reusable pieces of user interface used to build web applications. Components help divide large applications into smaller and manageable parts.
2. A component can represent different parts of the UI such as a header, footer, or student card. These components can be reused multiple times in different sections of an application.
3. React provides two types of components called class components and functional components. Both types are used for creating and displaying UI elements in React applications.
4. Component architecture in React means organizing the application into smaller reusable UI blocks. This architecture improves readability, maintenance, and code reusability.
5. Components communicate with each other using props which pass data from parent to child components. Props help share information between different parts of the application.
6. State is used inside components to store dynamic data that can change over time. When the state changes, React automatically updates the user interface.
7. An example of a functional component is function App() { return <h1>Hello</h1>; }. An example of a class component is class App extends Component { render() { return <h1>Hello</h1>; } }.
8. React component architecture makes application development faster and more organized. It also supports reusable code and efficient UI management in large projects.
9. Components work together to create complete user interfaces in React applications. This modular approach improves scalability and simplifies future updates in projects.

4 Describe class components with state and lifecycle methods.

1. Class components in React are ES6 classes used for creating user interfaces. They must extend React.Component to use React features properly.
2. A class component uses a compulsory render() method to display UI elements on the screen. The return statement inside render() defines what should appear in the browser.
3. Class components can manage dynamic data using state. State stores information that can change during application execution.
4. State values are updated using the setState() method inside class components. Whenever state changes, the user interface automatically re-renders with updated data.
5. Class components also support props for receiving data from parent components. Props are read-only and accessed using this.props.
6. Event handling can be implemented inside class components using functions. For example, a counter application can increase values using an increment function and setState().
7. Lifecycle methods are special methods that run during different stages of a component. They help perform tasks when components are created, updated, or removed from the application.
8. Earlier, lifecycle features were mainly available in class components before hooks were introduced. Hooks like useEffect now allow functional components to perform similar lifecycle operations.
9. Class components were widely used in older React applications for state and lifecycle management. They are still important for understanding the working and structure of React applications.

5 Explain state and props with examples and differences.

1. State and props are important concepts used for handling data in React applications. They help components store, manage, and transfer information efficiently.
2. Props are used to pass data from parent components to child components. Props are read-only and cannot be modified by child components.
3. An example of props is <Student name="Saloni" course="MCA" /> where data is passed to the Student component. Inside the component, values are accessed using this.props.name and this.props.course.
4. State is used to store dynamic data inside a component. State values can change during application execution and update the UI automatically.
5. In class components, state is updated using the setState() method. In functional components, the useState hook is used for state management.
6. An example of state is a counter application where the count value increases when a button is clicked. The updated state value is automatically displayed on the screen.
7. The main difference is that props are immutable while state is mutable. Props are passed from parent components, but state is managed within the same component.
8. Props help in communication between components while state handles changing data inside components. Both are important for creating interactive and dynamic React applications.
9. State and props together improve code organization and reusability in React projects. They help developers build scalable and user-friendly web applications efficiently.

6 Explain event handling and conditional rendering in React with examples.

1. Event handling in React is used to respond to user actions like clicks and typing. Conditional rendering is used to display UI based on specific conditions.
2. React uses camelCase syntax for event names such as onClick and onChange. Event handling functions are attached directly to JSX elements.
3. A button click event example is <button onClick={handleClick}>Click Me</button>. When the button is clicked, the handleClick function executes and performs an action like showing an alert.
4. Event handling can also update state values dynamically using setState() or useState(). This helps React applications respond interactively to user actions.
5. Conditional rendering displays different content depending on whether a condition is true or false. React supports conditional rendering using if statements, ternary operators, and the && operator.
6. An example of conditional rendering using an if statement is showing “Welcome User” when a user is logged in. Otherwise, the application displays “Please Login” to the user.
7. React also supports the ternary operator for shorter conditional rendering syntax. This method makes the code cleaner and easier to understand.
8. Event handling and conditional rendering together make React applications dynamic and interactive. They improve user experience by responding quickly to user actions and conditions.
9. These features are widely used in forms, login systems, dashboards, and modern web interfaces. They help developers create responsive and user-friendly applications efficiently.

7 Explain lists and forms in React with suitable examples.

1. Lists in React are collections of items usually stored inside arrays. They are used to display dynamic data such as student names, products, or messages.
2. React uses the map() function to display list items dynamically from arrays. The map() function loops through each element and creates UI elements like <li>.
3. Each item in a React list should contain a unique key attribute. The key helps React identify and update elements efficiently during rendering.
4. A simple example of a list is:
   {students.map((name, index) => (<li key={index}>{name}))}. This code displays all student names from an array in list format.
5. Forms in React are used to collect user input like login details, names, and registration data. Common form elements include input fields, textareas, dropdowns, and buttons.
6. React handles forms using events such as onChange and onSubmit. The event.target.value property is used to capture user input values.
7. Controlled components are forms where React manages form data using state. The input field value is connected to state using the value and onChange attributes.
8. An example of a React form is:
   <input type="text" value={name} onChange={handleChange} />. This updates the state automatically whenever the user types inside the input field.
9. Lists and forms help developers build dynamic and interactive applications in React. They improve data handling, user interaction, and overall user experience.

8 Explain React Hooks and advantages over class components.

1. React Hooks are special functions that allow functional components to use state and other React features. Hooks made functional components more powerful and flexible in React applications.
2. Earlier, only class components could use state and lifecycle features in React. Hooks now allow functional components to perform the same tasks easily.
3. The useState hook is used to store and update data inside functional components. It automatically updates the user interface whenever state values change.
4. The useEffect hook is used for side effects such as API calls, timers, and DOM updates. It runs automatically after the component renders on the screen.
5. Hooks simplify state management compared to class components. Developers can write shorter and cleaner code without using complex class syntax.
6. Hooks reduce the need for lifecycle methods used in class components. The useEffect hook performs many lifecycle-related tasks in a simpler way.
7. Functional components with hooks are easier to read and maintain than class components. This improves code organization and development speed in React projects.
8. Hooks improve code reusability because logic can be shared between components more easily. They also help create modern and scalable React applications efficiently.
9. React Hooks are now widely used in modern React development instead of class components. They provide better simplicity, readability, and performance for developers.

9 Explain useState and useEffect hooks with examples.

1. useState and useEffect are important React Hooks used in functional components. They allow developers to manage state and perform side effects easily.
2. The useState hook is used to store and update data inside functional components. Its syntax is const [state, setState] = useState(initialValue);.
3. useState automatically updates the user interface whenever the state value changes. It is commonly used for counters, forms, and dynamic data.
4. An example of useState is:
   const [count, setCount] = useState(0);. Clicking a button can increase the count value using setCount(count + 1).
5. The useEffect hook is used for side effects such as API calls, timers, and DOM updates. It runs automatically after the component renders.
6. The basic syntax of useEffect is useEffect(() => { }, []);. The empty dependency array means the code runs only once when the page loads.
7. useEffect is often used with fetch() to get data from backend APIs. The fetched data can then be stored using useState.
8. An example of useEffect is:
   useEffect(() => { fetch("http://localhost:8080/McaStudents") }, []);. This automatically calls the backend API when the component loads.
9. Both hooks simplify React development and reduce the need for class components. They help developers create cleaner, shorter, and more efficient React applications.

10 Develop a simple React application demonstrating state, event handling, and component interaction.

1. A simple React application can demonstrate state, event handling, and component interaction together. React applications are built using reusable components and dynamic data handling.
2. State is used to store dynamic values inside components. The useState hook helps functional components manage and update state easily.
3. Event handling is used to respond to user actions such as button clicks and typing. React uses events like onClick and onChange in camelCase format.
4. Component interaction happens when data is passed from a parent component to a child component using props. Props help components communicate with each other effectively.
5. Example parent component:
   <Student name="Rahul" />. Here, the parent component passes the name value to the Student child component using props.
6. Example child component:
   function Student(props) { return <h2>{props.name}</h2>; }. This component displays the received data on the screen.
7. Example of state and event handling:
   const [count, setCount] = useState(0);
   <button onClick={() => setCount(count + 1)}>Click</button>. Clicking the button updates the count value and re-renders the UI.
8. This application demonstrates how React handles dynamic updates and communication between components. It also shows how user actions can change the application interface instantly.
9. Such React applications are simple, interactive, and easy to maintain using components and hooks. They help developers create modern web applications with better user experience.